

Reporta tu Calle

Sprint 2: Patrones de Diseño

Integrantes:

1. Chontay Campos, Bryan Aldrin
2. Leon Gonzales, Paul Fran
3. Osorio Miranda, Arturo Miguel
4. Sánchez Longa, Erick Joel
5. Torres Reátegui, Joaquin Marcelo





- Anteriormente habíamos cubierto la función de reportar incidentes y mostrar una lista de esto

Ahora tomamos en cuenta las siguientes necesidades:

- La municipalidad necesita agrupar reportes duplicados
- Los supervisores requieren priorización y rutas de atención
- La evidencia multimedia debe gestionarse sin acoplar el dominio al almacenamiento

Núcleo del dominio

Reporte = incidente georreferenciado + categoría + evidencia + severidad ciudadana.

Decisión clave

La geolocalización no se calcula manualmente en Java: se delega a PostGIS.



REGISTRO COMPLETO

Reportes con categoría, imágenes y coordenadas precisas.

CONSULTA GEOGRÁFICA

Visualización pública de incidentes cercanos en el mapa.

ANTIDUPLICADOS

Consolidación inteligente de reportes en un radio de 20m.

RUTAS ÓPTIMAS

Optimización de recorridos por categoría de incidencia.

Seguridad robusta: Autenticación y autorización mediante tokens JWT.

ReportController.java

```
public class ReportController {  
  
    private final ReportService reportService;  
  
    @PostMapping  
    public ResponseEntity<ApiResponse<ReportResponse>> createReport(  
        @Valid @RequestBody CreateReportRequest request  
    ) {  
        ReportResponse response = reportService.createReport(request);  
        return ResponseEntity.status(HttpStatus.CREATED)  
            .body(ApiResponse.success(response, "Reporte ciudadano creado exitosamente"));  
    }  
  
    @GetMapping("/nearby")  
    public ResponseEntity<ApiResponse<List<ReportResponse>>> getReportsNearby(  
        @RequestParam Double latitude,  
        @RequestParam Double longitude,  
        @RequestParam(defaultValue = "5000") Double radius  
    ) {  
        List<ReportResponse> response = reportService.getReportsNearby(latitude, longitude, radius);  
  
        return ResponseEntity.ok(  
            ApiResponse.success(response, "Reportes cercanos recuperados exitosamente")  
        );  
    }  
}
```

Mantenibilidad

Módulos separados y puertos explícitos.

Testabilidad

Dominio y servicios probables con mocks.

Seguridad

API stateless con JWT y filtros transversales.

Rendimiento espacial

Índices GiST y consultas nativas PostGIS.

Evolución

Reemplazo de almacenamiento local por S3 sin tocar dominio.

Observabilidad de calidad

JaCoCo y SonarQube.

Trade-off aceptado

Más clases y mappers, pero menor acoplamiento tecnológico.

Criterio senior

La arquitectura protege los cambios probables: BD espacial, seguridad, storage, algoritmos.



- Dominio sin anotaciones JPA ni dependencias Spring
- Puertos de salida en domain para persistencia y storage
- Adaptadores implementan infraestructura concreta
- Controllers son adaptadores de entrada, no contienen negocio

ReportRepository.java · puerto de salida

```
/** PURO: Sin @Repository, sin extends JpaRepository, sin Spring */
public interface ReportRepository {
    Report save(Report report);
    Optional<Report> findById(Long id);
    List<Report> findReportsWithinRadius(Point center,
                                         double radiusInMeters);
}
```

Regla de dependencia

La capa domain define el contrato; infrastructure lo implementa.



✓ El filtro valida identidad antes de la capa web

⚙️ Controller delega y estandariza respuesta

🔍 Service aplica deduplicación y severidad

📍 Puerto consulta duplicados por radio

↔️ Adapter traduce dominio ⇌ JPA

🗄️ PostGIS ejecuta ST_DWithin

ReportService.java · deduplicación

```
Optional<Report> existingReportOpt =  
reportRepository.findExistingDuplicate(  
    request.categoryId(),  
    targetLocation,  
    DEDUPLICATION_RADIUS_METERS  
);
```

radio de consolidación

```
private static final double DEDUPLICATION_RADIUS_METERS = 20.0;
```

- **Patones:** Adapter, Facade, Proxy, Factory Method, Builder, Singleton gestionado por Spring
- **Bridge tecnológico:** AuthAccountJpaEntity implementa UserDetails
- **GoF implementado:** Decorator y Composite; no aparecen como estructura real en el backend
- **Strategy:** Aparece en optimización aunque no fue solicitado como foco principal

Mensaje para jurado

No vender patrones inexistentes. Se explica su ausencia como decisión de simplicidad.

Criterio técnico

Un patrón sólo se sustenta si resuelve un problema concreto y se puede navegar en el código.

Adapter

Facade

Proxy

Builder

Factory

Singleton

- El dominio Report no debe conocer JpaRepository, @Query ni entidades JPA
- PostGIS requiere consultas nativas y tipos geométricos
- El servicio de aplicación necesita trabajar con Report de dominio, no con ReportJpaEntity
- La alternativa directa era inyectar JpaRepository en ReportService

lo que NO se defendió

```
// Alternativa descartada
@Service
class ReportService {
    private final ReportJpaRepository repository;
    // Acopla aplicación a Hibernate/JPA
}
```

Decisión

Definir puerto ReportRepository en domain e implementarlo con ReportPersistenceAdapter.

SOLID

DIP: ReportService depende de una abstracción del dominio.

backend/src/main/java/com/reportatucalle/modules/report/domain/repository/ReportRepository.java

Puerto de salida

```
package com.reportatucalle.modules.report.domain.repository;

import com.reportatucalle.modules.report.domain.entity.Report;
import com.reportatucalle.modules.report.domain.entity.ReportStatus;
import org.locationtech.jts.geom.Point;
import java.util.List;
import java.util.Optional;

public interface ReportRepository {
    Report save(Report report);
    Optional<Report> findById(Long id);
    List<Report> findByStatus(ReportStatus status);
    List<Report> findByCitizenId(Long citizenId);
    List<Report> findReportsWithinRadius(Point center, double radiusInMeters);
    Optional<Report> findExistingDuplicate(Long categoryId,
                                         Point location,
                                         double radiusInMeters);
}
```

- Contrato puro del módulo report
- No extiende JpaRepository
- No usa @Repository
- Expresa necesidades del negocio: persistir, consultar por radio y detectar duplicado

Integración

ReportService inyecta ReportRepository; Spring resuelve la implementación concreta.

backend/src/main/java/com/reportatucalle/modules/report/infrastructure/persistence/adapter/ReportPersistenceAdapter.java

Adapter real Spring Boot

```
@Component
@RequiredArgsConstructor
public class ReportPersistenceAdapter implements ReportRepository {

    private final ReportJpaRepository jpaRepository;
    private final ReportPersistenceMapper mapper;

    @Override
    public Report save(Report report) {
        ReportJpaEntity jpaEntity = (report.getId() != null)
            ? mapper.toJpaEntity(report)
            : mapper.toJpaEntityForCreation(report);
        ReportJpaEntity saved = jpaRepository.save(jpaEntity);
        return mapper.toDomain(saved);
    }

    @Override
    public List<Report> findReportsWithinRadius(Point center,
                                                double radiusInMeters) {
        return jpaRepository.findReportsWithinRadius(center, radiusInMeters)
            .stream().map(mapper::toDomain).collect(Collectors.toList());
    }
}
```

Responsabilidad

Traducir llamadas del puerto a JPA y devolver dominio puro.

Trade-off

Mayor boilerplate por mapper, pero Hibernate no contamina el dominio.

Pregunta jurado

¿El mapper también es Adapter? No: el mapper es soporte; el adapter es quien implementa el puerto.

backend/src/main/java/com/reportatucalle/modules/report/infrastructure/persistence/repository/ReportJpaRepository.java

Infraestructura concreta

```
@Repository
public interface ReportJpaRepository extends JpaRepository<ReportJpaEntity, Long> {

    @Query(value = "SELECT * FROM reports r WHERE " +
        "ST_DWithin(CAST(r.location AS geography), " +
        "CAST(:center AS geography), :radiusInMeters)",
        nativeQuery = true)
    List<ReportJpaEntity> findReportsWithinRadius(
        @Param("center") Point center,
        @Param("radiusInMeters") double radiusInMeters);

    @Query(value = "SELECT * FROM reports r WHERE r.category_id = :categoryId " +
        "AND r.status IN ('PENDING', 'IN_PROGRESS') " +
        "AND ST_DWithin(CAST(r.location AS geography), " +
        "CAST(:location AS geography), :radiusInMeters) LIMIT 1",
        nativeQuery = true)
    Optional<ReportJpaEntity> findExistingDuplicate(...);
}
```

- Aquí sí aparece Spring Data JPA
- Aquí sí aparecen consultas nativas
- El adaptador oculta este detalle a ReportService
- La consulta usa ST_DWithin para radio geográfico

Clave técnica

PostGIS calcula distancia sobre geography, no una fórmula casera en Java.

backend/src/main/java/com/reportatucalle/modules/media/domain/portsout/StoragePort.java

StoragePort.java

```
public interface StoragePort {  
    String uploadFile(String fileName, byte[] fileData);  
    void deleteFile(String fileUrl);  
}
```

backend/src/main/java/com/reportatucalle/modules/media/infrastructure/adapter/LocalStorageAdapter.java

Adaptador local real

```
@Component  
@ConditionalOnProperty(name = "storage.provider",  
    havingValue = "local", matchIfMissing = true)  
public class LocalStorageAdapter implements StoragePort {  
  
    private final Path rootLocation = Paths.get("uploads");  
  
    @Override  
    public String uploadFile(String fileName, byte[] fileData) {  
        Path destinationFile = rootLocation.resolve(Paths.get(fileName))  
            .normalize().toAbsolutePath();  
        Files.write(destinationFile, fileData);  
        return baseUrl + "/uploads/" + fileName;  
    }  
}
```

- MediaService depende de StoragePort
- El proveedor se decide por propiedad de Spring
- S3/Cloudinary sería otro adapter
- No se toca el caso de uso al cambiar almacenamiento

Arquitectura

Puerto estable + adapters intercambiables.

- Módulos internos no deben navegar repositorios de otros módulos
- Optimization necesita datos de Category sin conocer su persistencia
- Report puede exponer operaciones pequeñas a otros módulos sin mostrar su servicio completo
- La alternativa era acoplar módulos a repositorios ajenos

acoplamiento inter-módulo

```
// Alternativa descartada
@Service
class RouteOptimizationService {
    private final CategoryRepository categoryRepository; //
    acoplamiento directo
}
```

Decisión

Cada módulo publica una API interna pequeña: CategoryFacade, ReportFacade, UserFacade.

SOLID

ISP + DIP: los consumidores ven sólo lo que necesitan.

backend/src/main/java/com/reportatucalle/modules/category/application/api/CategoryFacade.java

Contrato público del módulo Category

```
public interface CategoryFacade {
    Optional<AlgorithmType> getAlgorithmTypeForCategory(Long categoryId);
    Optional<String> getCategoryName(Long categoryId);
    boolean isCategoryActive(Long categoryId);
}
```

backend/src/main/java/com/reportatucalle/modules/category/application/api/CategoryFacadeImpl.java

Implementación real

```
@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class CategoryFacadeImpl implements CategoryFacade {
    private final CategoryRepository categoryRepository;

    @Override
    public Optional<AlgorithmType> getAlgorithmTypeForCategory(Long categoryId) {
        return categoryRepository.findById(categoryId)
            .map(category -> category.getAlgorithmType());
    }

    @Override
    public boolean isCategoryActive(Long categoryId) {
        return categoryRepository.findById(categoryId)
            .filter(category -> category.isAvailable()).isPresent();
    }
}
```

Uso real

RouteOptimizationService decide el algoritmo usando esta fachada.

Trade-off

Más contrato interno, pero menor dependencia transversal entre módulos.

backend/src/main/java/com/reportatucalle/modules/optimization/application/service/RouteOptimizationService.java

Facade + puertos de optimización

```
@Service
@RequiredArgsConstructor
public class RouteOptimizationService {

    private final CategoryFacade categoryFacade;
    private final RouteOptimizationPort routeOptimizationPort;
    private final FlowOptimizationPort flowOptimizationPort;
    private final ConnectivityOptimizationPort connectivityOptimizationPort;

    public Optional<OptimizedRoute> optimizeRoute(Long categoryId,
        Coordinate startPoint, List<Coordinate> destinations) {
        Optional<AlgorithmType> algorithmType =
            categoryFacade.getAlgorithmTypeForCategory(categoryId);
        if (algorithmType.isEmpty()) {
            throw new IllegalArgumentException("Categoría no encontrada: " + categoryId);
        }
        return executeStrategy(algorithmType.get(), startPoint, destinations);
    }
}
```

- Optimization no importa CategoryJpaRepository
- El algoritmo se decide por dato de dominio
- El módulo category puede cambiar internamente
- El servicio queda testeable con mock de CategoryFacade

Integración

Facade es el borde público del módulo, no una capa global.

backend/src/main/java/com/reportatucalle/modules/report/application/api/ReportFacadeImpl.java

Fachada pública del módulo Report

```
@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class ReportFacadeImpl implements ReportFacade {

    private final ReportRepository reportRepository;
    private final GeometryFactory geometryFactory =
        new GeometryFactory(new PrecisionModel(), 4326);

    @Override
    public Optional<Long> getReportIdIfExists(Long reportId) {
        return reportRepository.findById(reportId).map(report -> report.getId());
    }

    @Override
    public List<Long> getReportIdsNearby(double latitude, double longitude,
        double radiusInMeters) {
        Point location = geometryFactory.createPoint(new Coordinate(longitude, latitude));
        return reportRepository.findReportsWithinRadius(location, radiusInMeters)
            .stream().map(report -> report.getId()).collect(Collectors.toList());
    }
}
```

Problema resuelto

Otros módulos obtienen IDs o conteos sin conocer ReportService ni JPA.

Observación honesta

countReportsByCategory retorna 0L en el ZIP: no se debe vender como funcionalidad completa.

Defensa

El patrón existe; una operación está pendiente de completar.

- Los endpoints protegidos no deben ejecutar lógica si el usuario no está autenticado
- La validación JWT debe ser transversal y previa al controller
- La API es stateless: no se guarda sesión en servidor
- La alternativa era validar token manualmente en cada endpoint

lo que NO se hizo

```
// Alternativa descartada
@PostMapping
public ResponseEntity<?> createReport(@RequestHeader("Authorization")
String token) {
    // validación duplicada en cada endpoint
}
```

Tipo de Proxy

Protection Proxy: controla acceso antes del objeto real.



backend/src/main/java/com/reportatucalle/config/SecurityConfig.java

Configuración real Spring Security 6

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;
    private final AuthenticationProvider authenticationProvider;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(Customizer.withDefaults())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(PUBLIC_ENDPOINTS).permitAll()
                .anyRequest().authenticated())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authenticationProvider(authenticationProvider)
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

- PUBLIC_ENDPOINTS libera login, categorías, mapa cercano y uploads
- Todo lo demás requiere autenticación
- El filtro JWT corre antes del filtro usuario/contraseña
- La sesión se declara STATELESS

Trade-off

JWT escala bien, pero revocar tokens requiere estrategia adicional.

backend/src/main/java/com/reportatucalle/shared/security/JwtAuthenticationFilter.java

Proxy transversal

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
        HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String jwt = authHeader.substring(7);
        String userEmail = jwtService.extractUsername(jwt);
        if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails = userDetailsService.loadUserByUsername(userEmail);
            if (jwtService.isTokenValid(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                    userDetails, null, userDetails.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

Integración

Setea SecurityContextHolder antes de que ReportService use el usuario actual.

Riesgo controlado

Si el token falta o es inválido, no se autentica; endpoints protegidos rechazan.

SOLID

SRP: el filtro sólo autentica; no crea reportes ni valida negocio.

backend/src/main/java/com/reportatucalle/shared/security/JwtService.java

Servicio de emisión/validación JWT

```
@Service
public class JwtService {

    @Value("${app.security.jwt.secret}")
    private String secretKey;

    @Value("${app.security.jwt.expiration}")
    private long jwtExpiration;

    public String generateToken(Map<String, Object> extraClaims,
                               UserDetails userDetails) {
        return Jwts.builder()
            .claims(extraClaims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + jwtExpiration))
            .signWith(getSignInKey())
            .compact();
    }

    public boolean isValidToken(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
    }
}
```

- El filtro delega criptografía al servicio
- JwtService es stateless y administrado por Spring
- Los claims incluyen rol y profileId desde AuthService
- El token permite escalar sin sesión de servidor

Pregunta jurado

¿Cómo revocan tokens? Respuesta: blacklist/rotación corta/refresh tokens; no está implementado.

backend/src/main/java/com/reportatucalle/modules/auth/infrastructure/persistence/entity/AuthAccountJpaEntity.java

Bridge tecnológico real

```
@Entity
@Table(name = "auth_accounts")
@Getter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class AuthAccountJpaEntity implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 150)
    private String email;

    @Enumerated(EnumType.STRING)
    private Role role;

    // --- Spring Security UserDetails Bridge ---
    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority("ROLE_" + role.name()));
    }

    @Override
    public String getUsername() { return this.email; }
}
```

Problema

Spring Security exige UserDetails, pero el dominio AuthAccount debe permanecer puro.

Decisión

El puente se ubica en infraestructura, no en el dominio.

Advertencia

No venderlo como Bridge GoF canónico; es un puente de integración tecnológica.

backend/src/main/java/com/reportatucalle/shared/response/ApiResponse.java

Factory methods estáticos reales

```
@Getter
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ApiResponse<T> {

    private final boolean success;
    private final String message;
    private final T data;
    private final LocalDateTime timestamp;

    private ApiResponse(boolean success, String message, T data) {
        this.success = success;
        this.message = message;
        this.data = data;
        this.timestamp = LocalDateTime.now();
    }

    public static <T> ApiResponse<T> success(T data, String message) {
        return new ApiResponse<>(true, message, data);
    }

    public static <T> ApiResponse<T> success(String message) {
        return new ApiResponse<>(true, message, null);
    }

    public static <T> ApiResponse<T> error(String message) {
        return new ApiResponse<>(false, message, null);
    }
}
```

- Problema: respuestas API consistentes sin repetir constructores
- Constructor privado fuerza rutas válidas de creación
- Controller no decide estructura interna
- El timestamp se genera de forma uniforme

Trade-off

Es Factory Method estático, no una jerarquía polimórfica de factories.

backend/src/main/java/com/reportatucalle/modules/report/domain/entity/Report.java

Dominio inmutable

```
@Getter
public class Report {
    private final Long id;
    private final Long citizenId;
    private final Long categoryId;
    private final String title;
    private final String description;
    private final String imageUrl;
    private final Point location;
    private final ReportStatus status;
    private final Integer reportCount;
    private final LocalDateTime createdAt;
    private final LocalDateTime updatedAt;

    private Report(Builder builder) {
        this.id = builder.id;
        this.citizenId = builder.citizenId;
        this.categoryId = builder.categoryId;
        this.title = builder.title;
        this.description = builder.description;
        this.location = builder.location;
        this.status = builder.status;
        this.reportCount = builder.reportCount;
    }
}
```

Problema

Crear Report requiere muchos campos, defaults y validación fail-fast.

Decisión

Builder manual en dominio; Lombok sólo en otros objetos donde no se requiere control explícito.

SOLID

SRP: Report controla su construcción válida sin setters públicos.

Report.Builder · validación fail-fast

```
public static class Builder {
    private Long citizenId;
    private Long categoryId;
    private String title;
    private String description;
    private Point location;

    private ReportStatus status = ReportStatus.PENDING;
    private Integer reportCount = 1;
    private LocalDateTime createdAt = LocalDateTime.now();
    private LocalDateTime updatedAt = LocalDateTime.now();

    private void validate() {
        if (citizenId == null) throw new IllegalArgumentException("citizenId es requerido");
        if (categoryId == null) throw new IllegalArgumentException("categoryId es requerido");
        if (title == null || title.isBlank()) throw new IllegalArgumentException("title es requerido");
        if (description == null || description.isBlank()) throw new IllegalArgumentException("description es requerido");
        if (location == null) throw new IllegalArgumentException("location es requerido");
    }

    public Report build() {
        validate();
        return new Report(this);
    }
}
```

- Campos obligatorios se validan antes de crear instancia
- Valores por defecto: PENDING, reportCount=1, timestamps
- Evita objetos parcialmente contruidos
- Permite reconstruir Report al consolidar duplicados

Trade-off

Más código manual, pero mayor control sobre invariantes del dominio.

creación inicial

```
// ReportMapper.java
return Report.builder()
    .citizenId(citizenId)
    .categoryId(request.categoryId())
    .title(request.title())
    .description(request.description())
    .imageUrl(request.imageUrl())
    .location(location)
    .build();
```

- No se muta el objeto con setters
- Se crea una nueva instancia con severidad incrementada
- Se preserva el id y createdAt
- El adapter decide si hace INSERT o UPDATE

reconstrucción inmutable

```
// ReportService.java - consolidación de duplicado
existingReport = Report.builder()
    .id(existingReport.getId())
    .citizenId(existingReport.getCitizenId())
    .categoryId(existingReport.getCategoryId())
    .title(existingReport.getTitle())
    .description(existingReport.getDescription())
    .imageUrl(existingReport.getImageUrl())
    .location(existingReport.getLocation())
    .status(existingReport.getStatus())
    .reportCount(existingReport.getReportCount() + 1)
    .createdAt(existingReport.getCreatedAt())
    .updatedAt( LocalDateTime.now())
    .build();
```

Integración

Builder + Adapter trabajan juntos: dominio inmutable y persistencia actualizable.

JwtService.java

```
@Service
public class JwtService {
    @Value("${app.security.jwt.secret}")
    private String secretKey;

    @Value("${app.security.jwt.expiration}")
    private long jwtExpiration;

    public boolean isValidToken(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
    }
}
```

inyección del singleton

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private final JwtService jwtService;
    private final UserDetailsServiceImpl userDetailsService;
}
```

Problema

Servicios stateless no deben instanciarse manualmente por cada request.

Implementación

Spring crea un único bean singleton por ApplicationContext por defecto.

No se hizo

No se implementó Singleton clásico con getInstance(); se delegó al contenedor IoC.

backend/src/main/java/com/reportatucalle/config/ApplicationConfig.java

Beans singleton por defecto

```
@Configuration
@RequiredArgsConstructor
public class ApplicationConfig {

    private final AuthAccountJpaRepository authAccountJpaRepository;

    @Bean
    public UserDetailsService userDetailsService() {
        return username -> authAccountJpaRepository.findByEmail(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "Usuario no encontrado en la base de datos"));
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider =
            new DaoAuthenticationProvider(userDetailsService());
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

- UserDetailsService y PasswordEncoder se comparten como beans
- La configuración centraliza integración con Spring Security
- Spring permite sustituir beans en tests
- El ciclo de vida lo administra el ApplicationContext

Trade-off

Depender del contenedor exige levantar contexto en ciertos tests de integración.

backend/src/main/java/com/reportatucalle/modules/optimization/application/service/RouteOptimizationService.java

Selección de estrategia por AlgorithmType

```
private Optional<OptimizedRoute> executeStrategy(AlgorithmType algorithmType,
                                                Coordinate startPoint,
                                                List<Coordinate> destinations) {

    return switch (algorithmType) {
        case ROUTING -> {
            OptimizedRoute route = routeOptimizationPort
                .calculateOptimalRoute(startPoint, destinations);
            yield Optional.of(route);
        }
        case FLOW -> {
            Coordinate sink = destinations.get(destinations.size() - 1);
            List<Coordinate> network = destinations.subList(0, destinations.size() - 1);
            OptimizedRoute route = flowOptimizationPort.calculateMaxFlow(startPoint, sink, network);
            yield Optional.of(route);
        }
        case CONNECTIVITY -> {
            List<Coordinate> allNodes = new ArrayList<>();
            allNodes.add(startPoint); allNodes.addAll(destinations);
            yield Optional.of(connectivityOptimizationPort.calculateMinimumSpanningTree(allNodes));
        }
        case NONE -> Optional.empty();
    };
}
```

Valor

La categoría decide qué motor computacional se ejecuta.

Trade-off

Switch simple hoy; una factory/registry sería mejor si crecen muchos algoritmos.

Relación

CategoryFacade alimenta Strategy; puertos ejecutan algoritmos.

backend/src/main/java/com/reportatucalle/modules/optimization/domain/portsout/RouteOptimizationPort.java

Puerto

```
public interface RouteOptimizationPort {  
    OptimizedRoute calculateOptimalRoute(Coordinate startPoint,  
                                         List<Coordinate>  
                                         destinations);  
}
```

- El dominio pide ruta óptima vía puerto
- El adapter actual usa nearest neighbor
- Puede reemplazarse por OR-Tools/JSPRIT
- La app no queda atada al algoritmo actual

backend/src/main/java/com/reportatucalle/modules/optimization/infrastructure/adapter/RouteOptimizationAdapter.java

Adapter heurístico actual

```
@Component  
@RequiredArgsConstructor  
public class RouteOptimizationAdapter implements RouteOptimizationPort {  
    private static final double EARTH_RADIUS_KM = 6371.0;  
  
    @Override  
    public OptimizedRoute calculateOptimalRoute(Coordinate startPoint,  
                                                List<Coordinate> destinations) {  
        List<Coordinate> orderedStops = new ArrayList<>();  
        orderedStops.add(startPoint);  
        List<Coordinate> remaining = new ArrayList<>(destinations);  
        Coordinate current = startPoint;  
        double totalDistance = 0.0;  
  
        while (!remaining.isEmpty()) {  
            Coordinate nearest = remaining.get(0);  
            double minDistance = calculateDistance(current, nearest);  
            // nearest neighbor O(n²)  
            orderedStops.add(nearest);  
            totalDistance += minDistance;  
            remaining.remove(nearest);  
            current = nearest;  
        }  
        return new OptimizedRoute(orderedStops, totalDistance);  
    }  
}
```

Trade-off

Heurística rápida y simple; no garantiza óptimo global.

Decorator

No existe una cadena real de decoradores en backend. No se defiende como patrón implementado.

- Alternativa evaluada: decorar respuestas o media con validadores encadenados
- Se descartó porque MediaService ya concentra validación con reglas simples
- Forzarlo habría creado clases artificiales sin variabilidad real

Composite

No existe jerarquía uniforme parte-todo para reportes o recursos.

- El sistema maneja reportes y endorsements, no árboles de elementos homogéneos
- La relación reporte → evidencia es simple, no una estructura compuesta recursiva
- Se documenta como no aplicable al alcance actual

Respuesta honesta

pom.xml · stack espacial

```
<dependency>
  <groupId>net.postgis</groupId>
  <artifactId>postgis-jdbc</artifactId>
  <version>2024.1.0</version>
</dependency>
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-spatial</artifactId>
</dependency>
<dependency>
  <groupId>org.locationtech.jts</groupId>
  <artifactId>jts-core</artifactId>
</dependency>
```

- JTS representa Point en Java
- Hibernate Spatial persiste geometría
- PostGIS ejecuta funciones espaciales
- Docker Compose levanta la base con extensión incluida

docker-compose.yml

```
services:
  postgres-main:
    image: postgis/postgis:16-3.4
    container_name: rtc-postgres-main
    environment:
      POSTGRES_DB: reporta_tu_calle
    ports:
      - "5432:5432"
```

Decisión

Usar la base de datos para lo geoespacial, no lógica casera en application.

Caso de uso

```
// ReportService.java
private static final double DEDUPLICATION_RADIUS_METERS = 20.0;

Point targetLocation = geometryFactory.createPoint(
    new Coordinate(request.longitude(), request.latitude()));

Optional<Report> existingReportOpt = reportRepository.findExistingDuplicate(
    request.categoryId(),
    targetLocation,
    DEDUPLICATION_RADIUS_METERS
);
```

- Evita duplicados físicos para el mismo incidente
- reportCount modela severidad ciudadana
- El radio de 20m es decisión de negocio configurable
- El adapter mantiene limpia la regla de aplicación

ReportJpaRepository.java

```
@Query(value = "SELECT * FROM reports r WHERE r.category_id = :categoryId " +
    "AND r.status IN ('PENDING', 'IN_PROGRESS') " +
    "AND ST_DWithin(CAST(r.location AS geography), " +
    "CAST(:location AS geography), :radiusInMeters) " +
    "LIMIT 1", nativeQuery = true)
Optional<ReportJpaEntity> findExistingDuplicate(...);
```

Pregunta jurado

¿Por qué longitud primero? JTS usa X/Y: X=longitud, Y=latitud.

frontend/src/features/map/components/MapaReportes.jsx

Vertical slicing también en React

```
// Estructura del frontend por features
frontend/src/features/
├── auth/
├── categories/
├── map/
│   ├── components/MapaReportes.jsx
│   └── hooks/useGeolocalizacion.js
├── media/
├── reports/
│   ├── hooks/useCrearReporte.js
│   ├── hooks/useReportesCercanos.js
│   └── services/reportService.js
└── users/
```

- Leaflet renderiza mapa y marcadores
- Clustering evita degradación visual con muchos reportes
- Hooks encapsulan consumo de API y geolocalización
- AuthContext/Provider centraliza estado de autenticación

Relación backend

/reports/nearby alimenta el mapa; clustering es decisión de presentación.

backend/docker-compose.yml

servicios reales

```
services:
  postgres-main:
    image: postgres/postgis:16-3.4
    container_name: rtc-postgres-main
    ports:
      - "5432:5432"

  postgres-sonar:
    image: postgres:15
    container_name: rtc-postgres-sonar

  sonarqube:
    image: sonarqube:latest
    container_name: rtc-sonarqube
    depends_on:
      - postgres-sonar
    ports:
      - "9000:9000"
```

- Base principal PostGIS para reportes
- Base separada para SonarQube
- SonarQube aislado del runtime de negocio
- Volúmenes para persistencia local
- Adecuado para defensa y desarrollo local

Por qué no Kubernetes

Para un sprint universitario, Compose da reproducibilidad sin complejidad operativa innecesaria.

pom.xml · JaCoCo

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution><goals><goal>prepare-agent</goal></goals></execution>
    <execution>
      <id>report</id>
      <phase>verify</phase>
      <goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>
```

evidencia de tests

```
target/surefire-reports/
TEST-com.reportatucalle.modules.report.application.service.ReportServiceTest.xml
TEST-com.reportatucalle.modules.optimization.application.service.RouteOptimizationService
Test.xml
```

Cobertura JaCoCo

Líneas cubiertas: 633 / 903 ≈ 70.1% · Instrucciones: 2843 / 4051 ≈ 70.2%

- Pruebas unitarias por módulo: auth, category, report, media, optimization, user
- Mocks para puertos y facades
- Sonar excluye DTOs, config, entidades y respuestas del cálculo principal
- La cobertura no sustituye pruebas de comportamiento

- JWT sin revocación persistente: requiere refresh tokens o blacklist si crece seguridad
- countReportsByCategory en ReportFacadeImpl está pendiente
- GeometryFactory se instancia en varias clases: puede centralizarse como bean
- Nearest Neighbor no garantiza óptimo global
- Storage local no es escalable para producción; S3/Cloudinary vía StoragePort
- Decorator/Composite no deben defenderse como implementados

Fortaleza

La arquitectura contiene la deuda: cada riesgo tiene un punto de reemplazo claro.

Evolución natural

S3StorageAdapter, OptimizationRegistry, JWT refresh/blacklist, configuración de radio por properties.

Mensaje al jurado

No ocultar deuda técnica: demostrar que está localizada.

- 1. Levantar infraestructura: docker compose up -d
- 2. Ejecutar backend y validar Swagger
- 3. Login: obtener JWT
- 4. Crear reporte protegido con Bearer token
- 5. Consultar /reports/nearby sin login para mapa público
- 6. Crear reporte duplicado en radio de 20m y mostrar incremento de reportCount
- 7. Subir imagen y verificar /uploads/**
- 8. Ejecutar mvn test verify y mostrar JaCoCo/SonarQube

script corto

```
# comandos de defensa
cd backend
docker compose up -d
mvn clean verify
mvn spring-boot:run

# calidad
open target/site/jacoco/index.html
open http://localhost:9000
```

Cierre técnico

La demo debe mostrar patrones funcionando juntos: Proxy → Controller → Service → Adapter → PostGIS.

Última frase

No se eligieron patrones para decorar; se eligieron para proteger cambios probables del sistema.